

docs.google.com – 全画面表示を終了するには、Esc を押します

TL1

01_07.ファイル出力

日本工学院専門学校
デザインカレッジ

TL1

01_07.ファイル出力

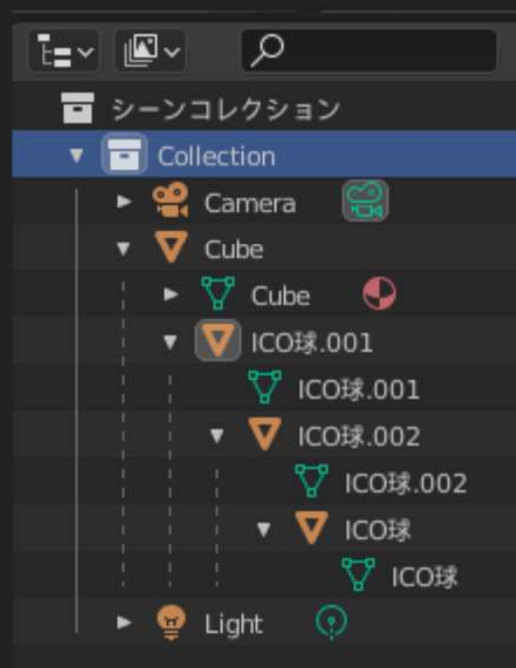
日本工学院専門学校
デザインカレッジ

実行結果

日本工学院専門学校
デザインカレッジ

出力したファイル

```
H: > sfujisawa > Downloads > untitled.scene
1 SCENE
2 MESH - Cube
3 Trans(0.000000,0.000000,0.000000)
4 Rot(0.000000,-0.000000,0.000000)
5 Scale(1.000000,1.000000,1.000000)
6
7 MESH - ICO球.001
8 Trans(0.000000,0.000000,0.000000)
9 Rot(0.000000,-0.000000,0.000000)
10 Scale(1.000000,1.000000,1.000000)
11
12 MESH - ICO球.002
13 Trans(0.000000,0.000000,0.000000)
14 Rot(0.000000,-0.000000,0.000000)
15 Scale(1.000000,1.000000,1.000000)
16
17 MESH - ICO球
18 Trans(0.000000,0.000000,0.000000)
19 Rot(0.000000,-0.000000,0.000000)
20 Scale(1.000000,1.000000,1.000000)
21
22 LIGHT - Light
23 Trans(4.076245,1.005454,5.903862)
24 Rot(37.261044,3.163707,106.936310)
25 Scale(1.000000,1.000000,1.000000)
26
27 CAMERA - Camera
28 Trans(7.358891,-6.925791,4.958309)
29 Rot(63.559303,-0.000003,46.691948)
30 Scale(1.000000,1.000000,1.000000)
```



ルートから順に出力していくことで、

オブジェクト同士の親子関係が
反映された出力になった。

この時点でも出力されたファイルは
最低限のシーンデータとして
使えなくもない。

ためしにゲームアプリ側でローダーを
書いてみて、ファイル書式に使いづらい点
があれば出力を変えてよい。
例えば .objに書式を寄せれば、
OBJローダーに近い感じでレベルローダー
を書ける。

字下げ

日本工学院専門学校
デザインカレッジ

parse_scene_recursive

```
def parse_scene_recursive(self, file, object, level):  
    """シーン解析用再帰関数"""  
  
    ...省略...  
  
    #子ノードへ進む(深さが1上がる)  
    for child in object.children:  
        self.parse_scene_recursive(file, child, level + 1)
```

カレントオブジェクト（今処理対象となるオブジェクト）の
直接の子供オブジェクトについて、同じ関数で処理する。（再帰呼び出し）

その際にlevel（深さ）を+1することで、インデントが1段階進む。

インデント

日本工学院専門学校
デザインカレッジ

parse_scene_recursive

```
def parse_scene_recursive(self, file, object, level):  
    """シーン解析用再帰関数"""  
  
    # 深さ分インデントする (タブを挿入)  
    indent = ''  
    for i in range(level):  
        indent += "\t"  
  
    # オブジェクト名書き込み  
    self.write_and_print(file, indent + object.type + " - " + object.name)  
    trans, rot, scale = object.matrix_local.decompose()  
    # 回転を Quaternion から Euler (3軸での回転角) に変換  
    rot = rot.to_euler()  
  
    # トランスフォーム情報を表示  
    self.write_and_print(file, indent + "Trans(%f,%f,%f)" %  
        trans)  
    self.write_and_print(file, indent + "Rot(%f,%f,%f)" %  
        rot)  
    self.write_and_print(file, indent + "Scale(%f,%f,%f)" %  
        scale)
```

親子関係を
インデントで
表現してみる。

子オブジェクトは
タブで字下げされ
るようになる。

実行結果

日本工学院専門学校
デザインカレッジ

出力したファイル

```
≡ untitled.scene ×  Untitled-1.cpp  ≡ ||  ?
C: > Users > fujisawa > Downloads > ≡ untitled.scene
1  SCENE
2  MESH - Cube
3  Trans(0.000000,0.000000,0.000000)
4  Rot(0.000000,-0.000000,0.000000)
5  Scale(1.000000,1.000000,1.000000)
6
7  LIGHT - Light
8  Trans(4.076245,1.005454,5.903862)
9  Rot(37.261044,3.163707,106.936310)
10 Scale(1.000000,1.000000,1.000000)
11
12 CAMERA - Camera
13 Trans(7.358891,-6.925791,4.958309)
14 Rot(63.559303,-0.000003,46.691948)
15 Scale(1.000000,1.000000,1.000000)
16
17 |
```

現時点では関数分けしただけなので、
結果は変わらない。

関数呼び出し

日本工学院専門学校
デザインカレッジ

export

```
#シーン内の全オブジェクトについて
for object in bpy.context.scene.objects:

    #親オブジェクトがあるものはスキップ（代わりに親から呼び出すから）
    if (object.parent):
        continue

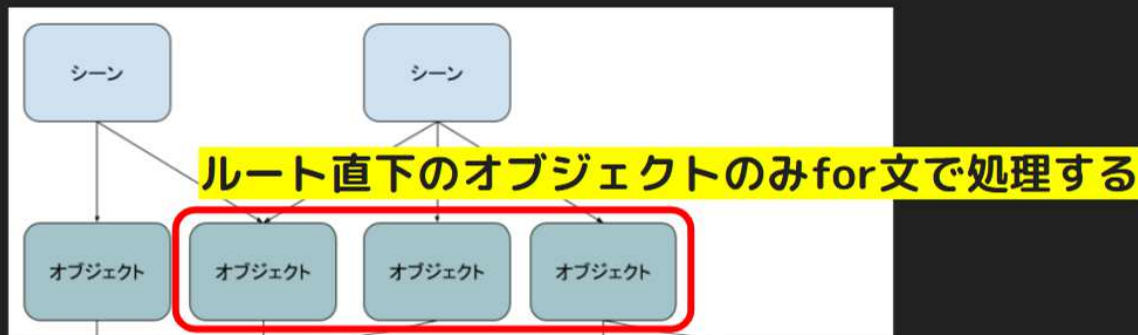
    #シーン直下のオブジェクトをルートノード（深さ0）とし、再帰関数で走査
    self.parse_scene_recursive(file, object, 0)
```

export() 内のfor文。

parse_scene_recursive()
にほとんど処理を移した。

parse_scene_recursive()
を再帰呼び出し関数に改造する
予定なので、

シーン構造でルート直下のもの
以外は、最初のfor文での処理
をcontinueでスキップする。



オブジェクト1個分の出力処理

日本工学院専門学校
デザインカレッジ

level_editor.py -class MYADDON_OT_export_scene

```
def parse_scene_recursive(self, file, object, level):
    """シーン解析用再帰関数"""

    # オブジェクト名書き込み
    self.write_and_print(file, object.type + " - " + object.name)
    trans, rot, scale = object.matrix_local.decompose()
    #回転を Quaternion から Euler (3軸での回転角) に変換
    rot = rot.to_euler()
    #ラジアンから度数法に変換
    rot.x = math.degrees(rot.x)
    rot.y = math.degrees(rot.y)
    rot.z = math.degrees(rot.z)
    #トランスフォーム情報を表示
    self.write_and_print(file, "Trans(%f,%f,%f)" % (trans.x, trans.y, trans.z) )
    self.write_and_print(file, "Rot(%f,%f,%f)" % (rot.x, rot.y, rot.z) )
    self.write_and_print(file, "Scale(%f,%f,%f)" % (scale.x, scale.y, scale.z) )
    self.write_and_print(file, '')
```

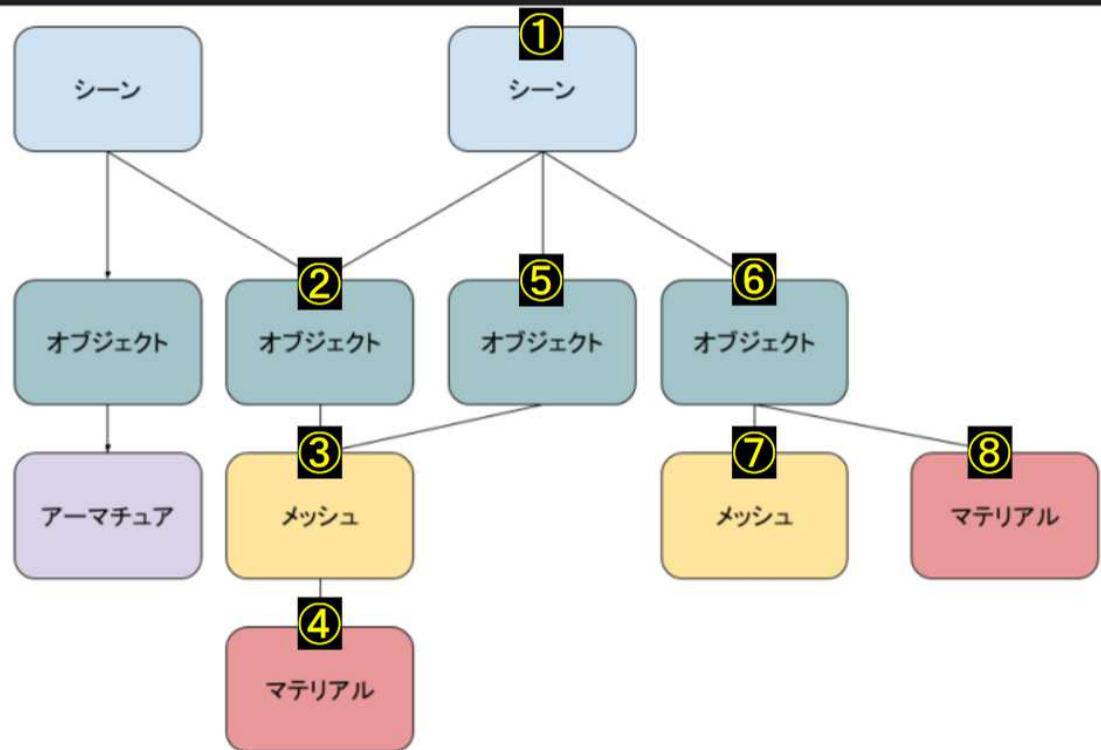
まず、関数分けを行う。

for文の中でやっていたオブジェクト1個分の情報出力処理を新たな関数として抜き出す。

親オブジェクト名表示は要らなくなるので消す。

ツリーを辿る

日本工学院専門学校
デザインカレッジ



シーングラフのイメージ

既に最低限のシーン情報は出力できたが、

シーン構造（シーングラフ）の特徴でもある木構造を辿って、根本であるシーンから順に出力する方式に変更してみたい。

今回は **深さ優先探索** を採用する。
(先へ先へ進む)

ツリーを辿る

実行結果

日本工学院専門学校
デザインカレッジ

出力したファイル

```
untitled.scene X  Untitled-1.cpp  ||  ?
C: > Users > fujisawa > Downloads > untitled.scene
1  SCENE
2  MESH - Cube
3  Trans(0.000000,0.000000,0.000000)
4  Rot(0.000000,-0.000000,0.000000)
5  Scale(1.000000,1.000000,1.000000)
6
7  LIGHT - Light
8  Trans(4.076245,1.005454,5.903862)
9  Rot(37.261044,3.163707,106.936310)
10 Scale(1.000000,1.000000,1.000000)
11
12 CAMERA - Camera
13 Trans(7.358891,-6.925791,4.958309)
14 Rot(63.559303,-0.000003,46.691948)
15 Scale(1.000000,1.000000,1.000000)
16
17 |
```

コンソール表示とファイル出力を両立しつつ、
改行が自動的に挿入されるようになった。

コンソールとファイルに同時出力

日本工学院専門学校
デザインカレッジ

```
level_editor.py -class MYADDON_OT_export_scene
```

```
def write_and_print(self, file, str):  
    print(str)  
  
    file.write(str)  
    file.write('\n')
```

\n (改行文字)を色んなところに入れるのは面倒なので、左のように関数にする。

file.write() に差し替えたところをさらにこの関数に差し替えることで、改行が自動的に挿入される。

ファイルハンドル

使い方

```
self.write_and_print(file, "あいうえお")
```

コンソールとファイルに同時出力

実行結果

日本工学院専門学校
デザインカレッジ

「無効化」→「有効化」で
アドオンを更新した後、
「シーン出力」を実行してみる。

コンソールウィンドウに表示していたシーン情報が
テキストファイルに書き込まれているのが確認できる。

ただし改行が入らず1行になってしまっている。

出力したファイル

≡ untitled.scene ×

C: > Users > fujisawa > Downloads > ≡ untitled.scene

```
1 SCENEMESH - CubeTrans(0.000000,0.000000,0.000000)Rot(0.000000,-0.000000,0.000000)Scale(1.000000,1.
```

ファイル書き出し

日本工学院専門学校
デザインカレッジ

```
level_editor.py -class MYADDON_OT_export_scene
```

```
def export(self):  
    """ファイルに出力"""  
  
    print("シーン情報出力開始... %r" % self.filepath)  
  
    #ファイルをテキスト形式で書き出し用にオープン  
    #スコープを抜けると自動的にクローズされる  
    with open(self.filepath, "wt") as file:  
  
        #ファイルに文字列を書き込む  
        file.write("SCENE")
```

ここに差し込む

10ページ でコメントアウトしておいた、
オブジェクト情報表示周りの処理を
export() の with ブロック内に差し込
む。

その上で print() だった部分を
file.write() に置き換える。

print() は自動的に最後に改行が入るが
file.write() は自動改行が入らないの
で、自分で \n (改行文字)を追加する。

例：

```
file.write("SCENE\n")
```

オブジェクトリスト書き出し

日本工学院専門学校
デザインカレッジ

いよいよ本命のオブジェクトリストを書き出していく。

これが上手くいけばシーンエディタとして最低限の機能は出来たと言ってもいいだろう。

オブジェクトリストをコンソールに表示させるのは既にやったので、出力先がコンソールなのかファイルなのかという違いはあるものの、概ね同じような処理で出来る。

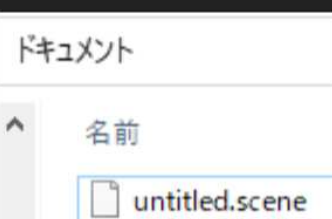
オブジェクトリスト書き出し

実行結果

日本工学院専門学校
デザインカレッジ

コンソール

```
シーン情報をExportします  
シーン情報出力開始... 'H:\sfujisawa\Documents\untitled.scene'  
シーン情報をExportしました
```



出力したファイル

```
H: > sfujisawa > Documents > untitled.scene  
1 SCENE  
2
```

「無効化」→「有効化」で
アドオンを更新した後、
「シーン出力」を実行してみる。

ExportHelper で選択したファイルパス
に実際にファイルが書き出されるので、
Windowsエクスプローラで確認する。

書き出された .scene ファイルは
テキストなので、VSCodeにドラッグアンド
ドロップして開く。

print() で指定した通りに "SCENE" という
文字列が書き込まれている。

関数呼び出し

日本工学院専門学校
デザインカレッジ

```
level_editor.py -class MYADDON_OT_export_scene
```

```
def execute(self, context):  
  
    print("シーン情報をExportします")  
  
    #ファイルに出力  
    self.export()  
  
    self.report({'INFO'}, "シーン情報をExportしました")  
    print("シーン情報をExportしました")  
  
    return {'FINISHED'}
```

`export()` は単なる自作関数なので、
自分で呼び出しを書かないと
実行されない。

ファイル出力のみテストしたいので、
`execute()` に書いていた処理の
大部分を一旦コメントアウトしておく。

python では
""" と """ で囲うことで、
複数行まとめてコメントアウトできる。
(ダブルクォーテーション3つずつ)
使ってみよう。

ファイル書き出し

日本工学院専門学校
デザインカレッジ

```
level_editor.py -class MYADDON_OT_export_scene
```

```
def export(self):  
    """ファイルに出力"""  
  
    print("シーン情報出力開始... %r" % self.filepath)  
  
    #ファイルをテキスト形式で書き出し用にオープン  
    #スコープを抜けると自動的にクローズされる  
    with open(self.filepath, "wt") as file:  
  
        #ファイルに文字列を書き込む  
        file.write("SCENE")
```

実際にファイルに書き出す処理を
新たなメンバ関数としてまとめる。

execute() が始まる時点で、
ExportHelper によって
`self.filepath` 変数に書き出し先となる
ファイルパスが代入されている。

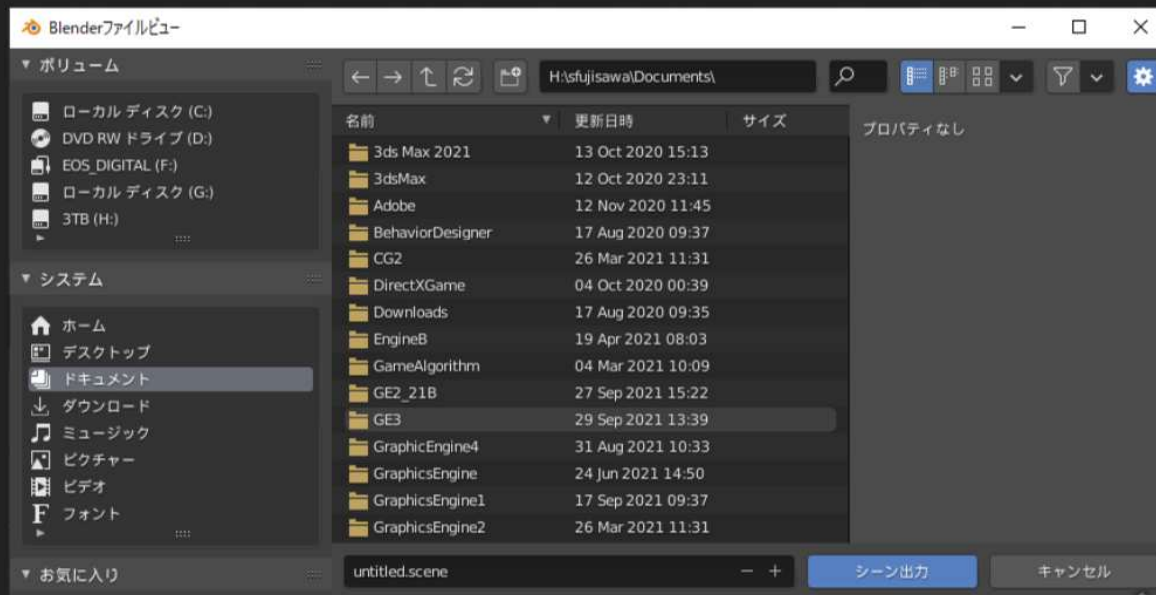
ファイルオープンなどの処理は
C++のfstream とそれほど変わらない。

ファイルパスを指定してオープンし、
ファイルハンドル `f` から `write()` を
呼び出して書き込む。

ファイル書き出し

実行結果

日本工学院専門学校
デザインカレッジ



前ページの処理だけで、
左図のようなファイル選択UIが
組み込まれる。

試しに「無効化」→「有効化」で
アドオンを更新した後、
「シーン出力」を実行してみることに。

確認してほしいのだが、まだ実際には
ファイルは出力されない。

ExportHelper が担当するのは、
書き出し先となるファイルパスを
決定するところまで。

実際の書き出しは別に処理を書く。

ExportHelperを継承

日本工学院専門学校
デザインカレッジ

```
import bpy_extras
```

```
#オペレータ シーン出力
class MYADDON_OT_export_scene(bpy.types.Operator, bpy_extras.io_utils.ExportHelper):
    bl_idname = "myaddon.myaddon_ot_export_scene"
    bl_label = "シーン出力"
    bl_description = "シーン情報をExportします"
    #出力するファイルの拡張子
    filename_ext = ".scene"
```

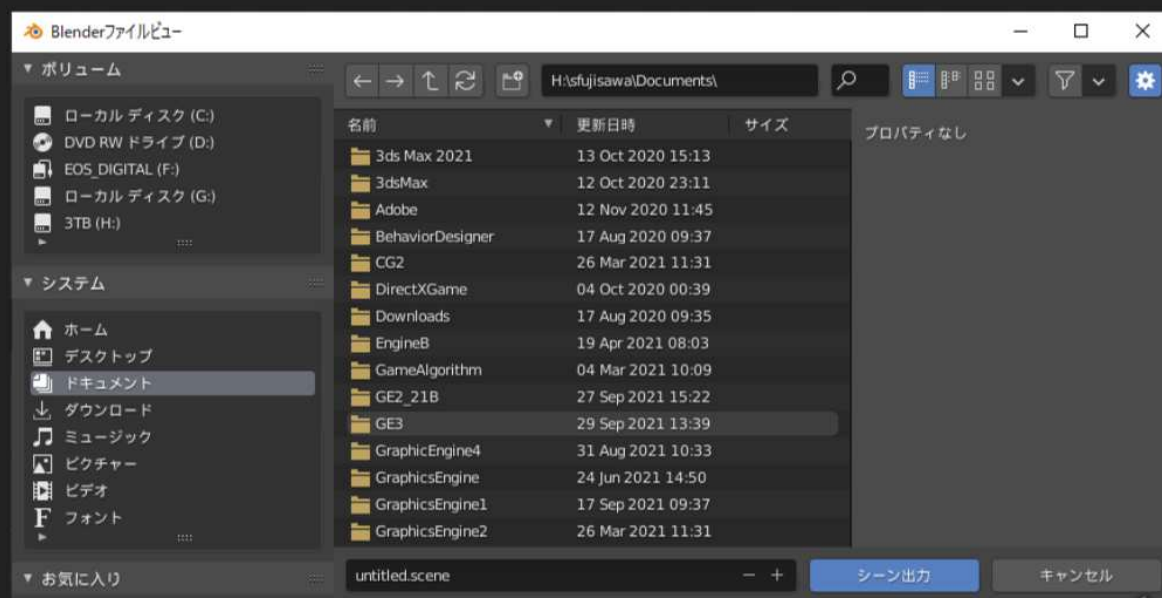
bpy_extras モジュールをインポートし、シーン出力用のクラスに多重継承する。
(オペレータ かつ エクスポートヘルパー なクラスになる)

ExportHelper を継承したら、「filename_ext」変数を定義しないとエラーになる。

この変数で、出力するファイルの拡張子を指定する。

ExportHelper

日本工学院専門学校
デザインカレッジ



既存のアドオンである
io_scene_fbx を参考に、
ExportHelper を継承する。

これを使うことで左図のような
グラフィックベースの
ファイル出力インターフェースを
利用できる。

リファレンスにも詳しくは載って
いないので、
使い方は他のアドオンを見て
分析するのがよいだろう。

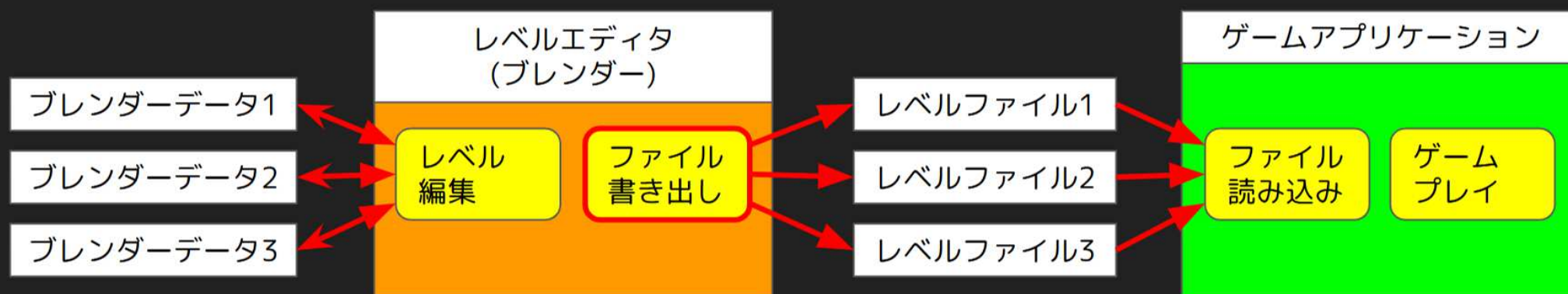
ExportHelper

ファイル出力

日本工学院専門学校
デザインカレッジ

前回、コンソールにシーンのオブジェクトリストを表示してみたが
ゲームで使えるシーンデータを作るのが目標なので、
シーンの情報をファイルとして出力する必要がある。

それをやっていきたい。



今回の目標

日本工学院専門学校
デザインカレッジ

- シーン上のオブジェクトリストをファイルに出力する

今回の目標

日本工学院専門学校
デザインカレッジ

- シーン上のオブジェクトリストをファイルに出力する